

KLE SOCIETY'S COLLEGE OF SCIENCE & COMMERCE

NAAC Accredited Grade B+

(Affiliated to University of Mumbai)

Plot No. 29, Sector No. 01, Kalambohi, Navi Mumbai

(2025-2026)



DEPARTMENT OF INFORMATION TECHNOLOGY

PRACTICAL BOOK

ON

DEEP LEARNING

Submitted To

UNIVERSITY OF MUMBAI

BY

MAHESH TANAJI CHAVAN

Seat No: -1314518



**MASTER OF SCIENCE INFORMATION
TECHNOLOGY PART – II (SEMESTER - IV)
(2025-2026)**

KLE SOCIETY'S COLLEGE OF SCIENCE & COMMERCE

NAAC Accredited Grade B+

(Affiliated to University of Mumbai)

**Plot No. 29, Sector No. 01, Kalamboli, Navi Mumbai
(2025-2026)**



CERTIFICATE

This is to certify that

Mr./Ms. _____

of Class _____, Semester _____

has successfully completed the practical for the subject _____

in M.Sc. (Information Technology) during the academic year 2025- 2026 as per the
syllabus prescribed by the University of Mumbai.

Prof. In charge

Date: _____

Head of the Department

Date: _____

External Examiner

Date: _____

Principal Sign

Date: _____

MAHESH TANAJI CHAVAN

College Stamp
Msc.IT (Part II)

Seat No:1314518

INDEX

SR No.	Practical List	Date	Sign
1	Introduction to TensorFlow		
a.	• Create tensors with different shapes and data types. • Perform basic operations like addition, subtraction, multiplication, and division on tensors. • Reshape, slice, and index tensors to extract specific elements or sections • Performing matrix multiplication and finding eigenvectors and eigenvalues using TensorFlow		
b.	Program to solve the XOR problem		
2	Linear Regression		
a.	• Implement a simple linear regression model using TensorFlow's lowlevel API (or tf. keras). • Train the model on a toy dataset (e.g., housing prices vs. square footage) • Visualize the loss function and the learned linear relationship.		
b.	Make predictions on new data points		
3	Convolutional Neural Networks (Classification)		
a.	Implementing deep neural network for performing binary classification task		
b.	Using a deep feed-forward network with two hidden layers for performing multiclass classification and predicting the class.		
4	Write a program to implement deep learning Techniques for image segmentation.		
5	Write a program to predict a caption for a sample image using LSTM		
6	Applying the Autoencoder algorithms for encoding real-world data		
7	Write a program for character recognition using RNN and compare it with CNN.		
8	Write a program to develop Autoencoders using MNIST Handwritten Digits		

9	Demonstrate recurrent neural network that learns to perform sequence analysis for stock price. (google stock price)		
10	Applying Generative Adversarial Networks for image generation and unsupervised tasks.		

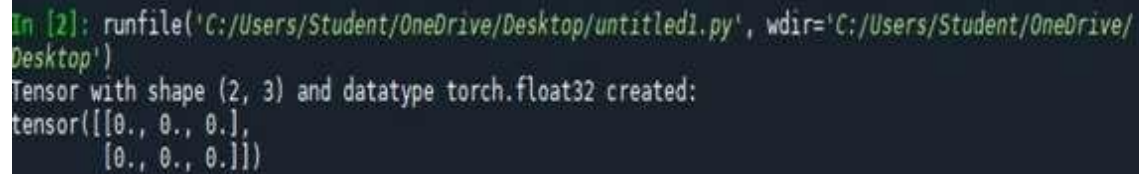
Practical 1

Introduction to TensorFlow

1.a.a Create tensors with different shapes and data types

```
# Import PyTorch
import torch
def create_tensor(shape, dtype=torch.float32):
    # Create tensor with given shape and datatype
    tensor = torch.zeros(shape, dtype=dtype)
    print(f"Tensor with shape {shape} and datatype {dtype} created:")
    print(tensor)
    return tensor
demo_shape = (2, 3)
demo_dtype = torch.float32
create_tensor(demo_shape, demo_dtype)
```

OUTPUT

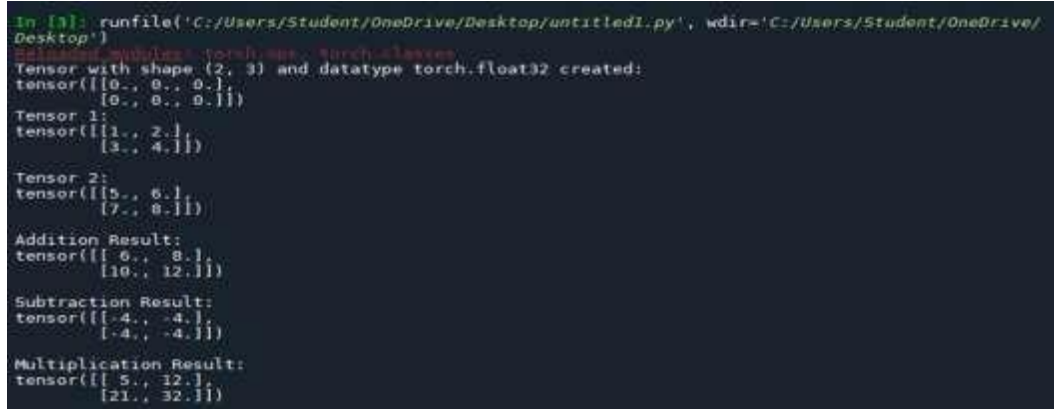


```
In [2]: runfile('C:/Users/Student/OneDrive/Desktop/untitled1.py', wdir='C:/Users/Student/OneDrive/Desktop')
Tensor with shape (2, 3) and datatype torch.float32 created:
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

1.a.b. Perform basic operations like addition, subtraction, multiplication, and division on tensors

```
# Import PyTorch
import torch
def create_tensor(shape, dtype=torch.float32):
    tensor = torch.zeros(shape, dtype=dtype)
    print(f'Tensor with shape {shape} and datatype {dtype} created:')
    print(tensor)
    return tensor
def tensor_operations():
    tensor1 = torch.tensor([[1, 2], [3, 4]], dtype=torch.float32)
    tensor2 = torch.tensor([[5, 6], [7, 8]], dtype=torch.float32)
    print('Tensor 1:')
    print(tensor1)
    print('\nTensor 2:')
    print(tensor2)
    addition_result = torch.add(tensor1, tensor2)
    print('\nAddition Result:')
    print(addition_result)
    subtraction_result = torch.sub(tensor1, tensor2)
    print('\nSubtraction Result:')
    print(subtraction_result)
    multiplication_result = torch.mul(tensor1, tensor2)
    print('\nMultiplication Result:')
    print(multiplication_result)
    create_tensor((2, 3))
    tensor_operations()
```

OUTPUT



```
In [3]: runfile('C:/Users/Student/OneDrive/Desktop/untitled1.py', wdir='C:/Users/Student/OneDrive/Desktop')
Tensor with shape (2, 3) and datatype torch.float32 created:
tensor([[0., 0., 0.],
        [0., 0., 0.]])
Tensor 1:
tensor([[1., 2.],
        [3., 4.]])
Tensor 2:
tensor([[5., 6.],
        [7., 8.]])
Addition Result:
tensor([[ 6.,  8.],
        [10., 12.]])
Subtraction Result:
tensor([[ -4., -4.],
        [ -4., -4.]])
Multiplication Result:
tensor([[ 5., 12.],
        [21., 32.]])
```

a.a. Reshape, slice, and index tensors to extract specific elements or sections

```
# Import PyTorch
import os
os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'
import torch
def create_tensor(shape, dtype=torch.float32):
    tensor = torch.arange(shape[0] * shape[1], dtype=dtype).reshape(shape)
    print(f'Original Tensor with shape {shape} and datatype {dtype}:')
    print(tensor)
    return tensor
def perform_operations(tensor):
    # Reshape the tensor
    reshaped_tensor = tensor.reshape(-1, 2)
    print('\nReshaped Tensor (-1, 2):')
    print(reshaped_tensor)
    # Slice the tensor (Extract 1st row and 1st 2 columns)
    sliced_tensor = tensor[0, :2]
    print('\nSliced Tensor (1st row, 1st 2 columns):')
    print(sliced_tensor)
    # Extract specific element (e.g., element at (1,1))
    specific_element = tensor[1, 1]
    print('\nSpecific Element at (1,1):')
    print(specific_element)
    # Example: Create a 2x3 tensor of float32
    demo_shape = (2, 3)
    demo_dtype = torch.float32
    tensor = create_tensor(demo_shape, demo_dtype)
    # Perform reshape, slice, and extract operations
    perform_operations(tensor)
```

OUTPUT



```
In [1]: runfile('C:/Users/Student/.spyder-py3/temp.py', wdir='C:/Users/Student/.spyder-py3')
Original Tensor with shape (2, 3) and datatype torch.float32:
tensor([[0., 1., 2.],
        [3., 4., 5.]])

Reshaped Tensor (-1, 2):
tensor([[0., 1.],
        [2., 3.],
        [4., 5.]])

Sliced Tensor (1st row, 1st 2 columns):
tensor([0., 1.])

Specific Element at (1,1):
tensor(4.)
```

a.b.d Performing matrix multiplication and finding eigenvectors and eigenvalues using TensorFlow

```
# Import TensorFlow
import tensorflow as tf
# Perform Matrix Multiplication
def matrix_multiplication(A, B):
    result = tf.linalg.matmul(A, B)
    print("Matrix Multiplication Result:")
    print(result.numpy())
# Find Eigenvalues and Eigenvectors
def find_eigen(A):
    eigenvalues, eigenvectors = tf.linalg.eigh(A)
    print("Eigenvalues:")
    print(eigenvalues.numpy())
    print("Eigenvectors:")
    print(eigenvectors.numpy())
# Example Matrices (2x2)
A = tf.constant([[4.0, -2.0], [1.0, 1.0]], dtype=tf.float32)
B = tf.constant([[1.0, 2.0], [3.0, 4.0]], dtype=tf.float32)
# Perform Operations
matrix_multiplication(A, B)
find_eigen(A)
```

OUTPUT



```
In [1]: runfile('C:/Users/Student/OneDrive/Desktop/untitled0.py', wdir='C:/Users/Student/OneDrive/Desktop')
Matrix Multiplication Result:
[[-2.  0.]
 [ 4.  6.]]
Eigenvalues:
[0.6972243  4.3027754]
Eigenvectors:
[[-0.28978416  0.957092 ]
 [ 0.957092   0.28978416]]
```


1.b. Program to solve the XOR problem.

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
# XOR Input and Output Data
x_train = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_train = np.array([[0], [1], [1], [0]])
# Define a simple Neural Network model
model = keras.Sequential([
    layers.Dense(4, activation='relu', input_shape=(2,)),
    layers.Dense(1, activation='sigmoid')
])
# Compile the model
model.compile(optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy'])
# Train the model
model.fit(x_train, y_train, epochs=500, verbose=0)
# Evaluate the model
loss, accuracy = model.evaluate(x_train, y_train)
print(f"Training Accuracy: {accuracy * 100:.2f}%")
# Predict the outputs for XOR inputs
predictions = model.predict(x_train)
print("\nPredictions:")
for i, pred in enumerate(predictions):
    print(f"Input: {x_train[i]} => Predicted: {round(pred[0])}, Actual: {y_train[i][0]}")
```

OUTPUT

```
In [1]: runfile('C:/Users/student/OneDrive/Desktop/untitled1.py', wdir='C:/Users/student/OneDrive/Desktop')
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11.
Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
C:\Users\Student\anaconda3\lib\site-packages\keras\src\layers\core\dense.py:106: UserWarning: Do not pass an 'input_shape' / 'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 ----- 0s 96ms/step - accuracy: 1.0000 - loss: 0.3970
Training Accuracy: 100.00%
1/1 ----- 0s 48ms/step

Predictions:
Input: [0 0] => Predicted: 0, Actual: 0
Input: [0 1] => Predicted: 1, Actual: 1
Input: [1 0] => Predicted: 1, Actual: 1
Input: [1 1] => Predicted: 0, Actual: 0
```

Practical 2

Linear Regression

2.a Implement a simple linear regression model using TensorFlow's low level API (or tf.keras).

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
X_train = np.linspace(0, 10, 100).reshape(-1, 1).astype(np.float32)
y_train = 2.5 * X_train + np.random.normal(0, 2, (100, 1)).astype(np.float32) #
y = 2.5x + noise
W = tf.Variable(tf.random.normal([1, 1], dtype=tf.float32))
b = tf.Variable(tf.random.normal([1], dtype=tf.float32))
def linear_regression(X):
    return tf.matmul(X, W) + b
def loss_fn(y_true, y_pred):
    return tf.reduce_mean(tf.square(y_true - y_pred))
optimizer = tf.optimizers.SGD(learning_rate=0.01)
def train_step(X, y):
    with tf.GradientTape() as tape:
        y_pred = linear_regression(X)
        loss = loss_fn(y, y_pred)
        gradients = tape.gradient(loss, [W, b])
        optimizer.apply_gradients(zip(gradients, [W, b]))
    return loss
epochs = 200
loss_history = []
for epoch in range(epochs):
    loss = train_step(X_train, y_train)
    loss_history.append(loss.numpy())
    if epoch % 20 == 0:
        print(f'Epoch {epoch}, Loss: {loss.numpy():.4f}')
        plt.plot(range(epochs), loss_history, label='Loss')
        plt.xlabel('Epochs')
        plt.ylabel('Loss')
        plt.title('Training Loss')
        plt.legend()
        plt.show()
    plt.scatter(X_train, y_train, label='Data')
    plt.plot(X_train, linear_regression(X_train).numpy(), color='red', label='Fitted Line')
    plt.xlabel('X')
```

```
plt.ylabel('#y#')
plt.title('#Linear Regression Fit#')
plt.legend()
plt.show()
print(f'Trained Weight: {W.numpy()[0][0]:.4f}, Bias: {b.numpy()[0]:.4f}')
```

OUTPUT

```
In [1]: runfile('C:/Users/Student/OneDrive/Desktop/untitled1.py', wdir='C:/Users/Student/OneDrive/
Desktop')
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11.
Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML
plugin.
C:\Users\Student\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:106: UserWarning: Do not
pass an 'input_shape' / 'input_dim' argument to a layer. When using Sequential models, prefer using an
'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
1/1 ----- 0s 96ms/step - accuracy: 1.0000 - loss: 0.3970
Training Accuracy: 100.00%
1/1 ----- 0s 48ms/step

Predictions:
Input: [0 0] => Predicted: 0, Actual: 0
Input: [0 1] => Predicted: 1, Actual: 1
Input: [1 0] => Predicted: 1, Actual: 1
Input: [1 1] => Predicted: 0, Actual: 0
```

2.b Train the model on a toy dataset (e.g., housing prices vs. square footage).

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
house_size = np.linspace(500, 4000, 100).reshape(-1, 1).astype(np.float32) #
House size in sq ft
house_price = 50 + 0.1 * house_size + np.random.normal(0, 10, (100,
1)).astype(np.float32) # Price in $1000s
house_size_norm = (house_size - np.mean(house_size)) / np.std(house_size)
house_price_norm = (house_price - np.mean(house_price)) /
np.std(house_price)
W = tf.Variable(tf.random.normal([1, 1], dtype=tf.float32))
b = tf.Variable(tf.random.normal([1], dtype=tf.float32))
def linear_regression(X):
    return tf.matmul(X, W) + b
def loss_fn(y_true, y_pred):
    return tf.reduce_mean(tf.square(y_true - y_pred))
optimizer = tf.optimizers.SGD(learning_rate=0.01)
def train_step(X, y):
    with tf.GradientTape() as tape:
        y_pred = linear_regression(X)
        loss = loss_fn(y, y_pred)
        gradients = tape.gradient(loss, [W, b])
        optimizer.apply_gradients(zip(gradients, [W, b]))
    return loss
epochs = 200
loss_history = []
for epoch in range(epochs):
    loss = train_step(house_size_norm, house_price_norm)
    loss_history.append(loss.numpy())
    if epoch % 20 == 0:
        print(f'Epoch {epoch}, Loss: {loss.numpy():.4f}')
plt.plot(range(epochs), loss_history, label='Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training Loss')
plt.legend()
plt.show()
plt.scatter(house_size_norm, house_price_norm, label='Data')
plt.plot(house_size_norm, linear_regression(house_size_norm).numpy(),
color='red', label='Fitted Line')
plt.xlabel('Normalized House Size')
```

```
plt.ylabel('#39;Normalized Price#39;)
plt.title('#39;Linear Regression Fit#39;)
plt.legend()
plt.show()
print(f'&quot;Trained Weight: {W.numpy()[0][0]:.4f}, Bias: {b.numpy()[0]:.4f}&quot;)
```

OUTPUT

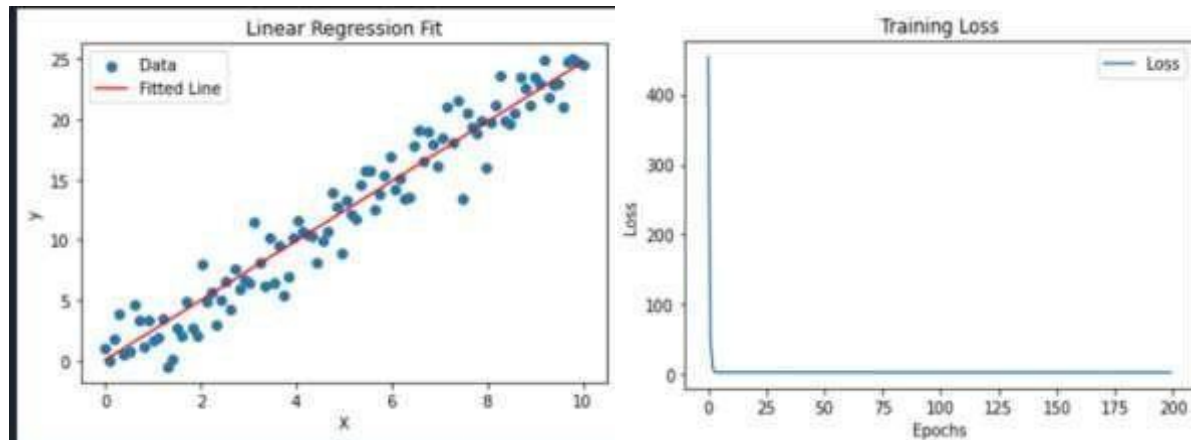
```
in [4]: runfile('C:/Users/Student/OneDrive/Desktop/untitled1.py', wdir='C:/Users/Student/OneDrive/
Desktop')
Epoch 0, Loss: 453.0059
Epoch 20, Loss: 3.4509
Epoch 40, Loss: 3.4230
Epoch 60, Loss: 3.3935
Epoch 80, Loss: 3.3693
Epoch 100, Loss: 3.3495
Epoch 120, Loss: 3.3333
Epoch 140, Loss: 3.3200
Epoch 160, Loss: 3.3091
Epoch 180, Loss: 3.3002
```

Important

Figures are displayed in the Plots pane by default. To make them also appear inline in the console, you need to uncheck "Mute inline plotting" under the options menu of Plots.

Trained Weight: 2.4732, Bias: 0.0162

PLOTS



2.c Visualize the loss function and the learned linear relationship.

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(42)
house_size = np.linspace(500, 4000, 100).reshape(-1, 1).astype(np.float32) #
House size in sq ft
house_price = 50 + 0.1 * house_size + np.random.normal(0, 10, (100,
1)).astype(np.float32) # Price in $1000s
house_size_norm = (house_size - np.mean(house_size)) / np.std(house_size)
house_price_norm = (house_price - np.mean(house_price)) /
np.std(house_price)
W = tf.Variable(tf.random.normal([1, 1], dtype=tf.float32))
b = tf.Variable(tf.random.normal([1], dtype=tf.float32))
def linear_regression(X):
    return tf.matmul(X, W) + b
def loss_fn(y_true, y_pred):
    return tf.reduce_mean(tf.square(y_true - y_pred))
optimizer = tf.optimizers.SGD(learning_rate=0.01)
def train_step(X, y):
    with tf.GradientTape() as tape:
        y_pred = linear_regression(X)
        loss = loss_fn(y, y_pred)
        gradients = tape.gradient(loss, [W, b])
        optimizer.apply_gradients(zip(gradients, [W, b]))
    return loss
epochs = 200
loss_history = []
for epoch in range(epochs):
    loss = train_step(house_size_norm, house_price_norm)
    loss_history.append(loss.numpy())
    if epoch % 20 == 0:
        print(f'Epoch {epoch}, Loss: {loss.numpy():.4f}')
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(range(epochs), loss_history, label='Loss', color='blue')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training Loss')
plt.legend()
plt.subplot(1, 2, 2)
plt.scatter(house_size, house_price, label='Data', color='gray')
predicted_price = linear_regression(house_size_norm).numpy() *
```

```

np.std(house_price) + np.mean(house_price)
plt.plot(house_size, predicted_price, color='red', label='Fitted Line')
plt.xlabel('House Size (sq ft)')
plt.ylabel('House Price ($1000s)')
plt.title('Learned Linear Relationship')
plt.legend()
plt.show()
print(f'Trained Weight: {W.numpy()[0][0]:.4f}, Bias: {b.numpy()[0]:.4f}')

```

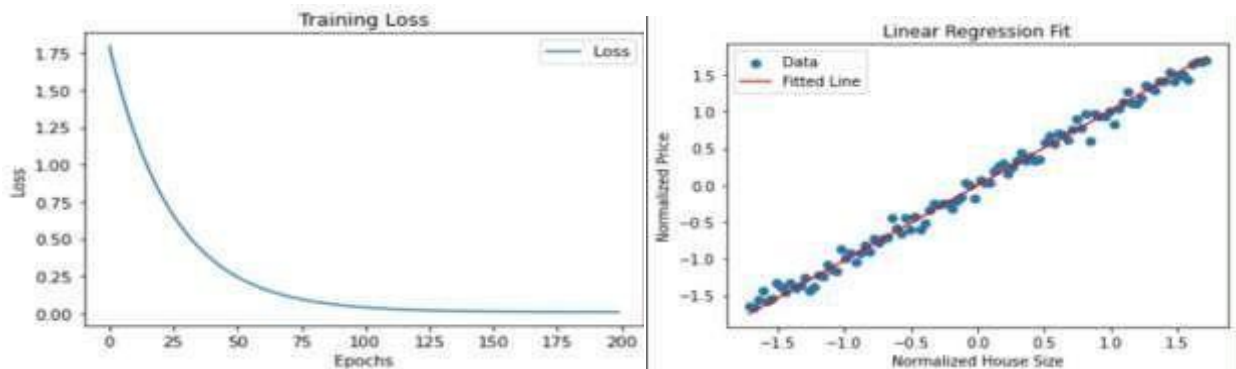
OUTPUT

```

In [5]: runfile('C:/Users/Student/OneDrive/Desktop/untitled1.py', wdir='C:/Users/Student/OneDrive/Desktop')
Epoch 0, Loss: 1.7946
Epoch 20, Loss: 0.8041
Epoch 40, Loss: 0.3627
Epoch 60, Loss: 0.1659
Epoch 80, Loss: 0.0782
Epoch 100, Loss: 0.0391
Epoch 120, Loss: 0.0217
Epoch 140, Loss: 0.0139
Epoch 160, Loss: 0.0105
Epoch 180, Loss: 0.0089
Trained Weight: 1.0192, Bias: -0.0047

```

PLOTS



2.d Make predictions on new data points.

```
# Install required library (only for Jupyter/Colab)
# !pip install tensorflow tensorflow-datasets -q

# Import libraries
import tensorflow as tf
import tensorflow_datasets as tfds
import numpy as np
import matplotlib.pyplot as plt

# =====
# PART 1: Linear Regression
# =====

# Generate dataset
np.random.seed(42)
house_size = np.linspace(500, 4000, 100).reshape(-1, 1).astype(np.float32) # sq ft
house_price = 50 + 0.1 * house_size + np.random.normal(0, 10, (100, 1)).astype(np.float32)
# Normalize data
house_size_mean = np.mean(house_size)
house_size_std = np.std(house_size)
house_price_mean = np.mean(house_price)
house_price_std = np.std(house_price)
house_size_norm = (house_size - house_size_mean) / house_size_std
house_price_norm = (house_price - house_price_mean) / house_price_std
# Model parameters
W = tf.Variable(tf.random.normal([1, 1], dtype=tf.float32))
b = tf.Variable(tf.random.normal([1], dtype=tf.float32))
# Model
def linear_regression(X):
    return tf.matmul(X, W) + b
# Loss
def loss_fn(y_true, y_pred):
    return tf.reduce_mean(tf.square(y_true - y_pred))
# Optimizer
optimizer = tf.optimizers.SGD(learning_rate=0.01)
# Training step
def train_step(X, y):
    with tf.GradientTape() as tape:
        y_pred = linear_regression(X)
        loss = loss_fn(y, y_pred)
    gradients = tape.gradient(loss, [W, b])
    optimizer.apply_gradients(zip(gradients, [W, b]))
```



```

    return loss
# Training loop
epochs = 200
loss_history = []
for epoch in range(epochs):
    loss = train_step(house_size_norm, house_price_norm)
    loss_history.append(loss.numpy())
    if epoch % 20 == 0:
        print(f"Epoch {epoch}, Loss: {loss.numpy():.4f}")
# Plot results
plt.figure(figsize=(12, 5))
# Loss plot
plt.subplot(1, 2, 1)
plt.plot(range(epochs), loss_history, label='Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training Loss')
plt.legend()
# Regression plot
plt.subplot(1, 2, 2)
plt.scatter(house_size, house_price, color='gray', label='Data')
pred_line = (linear_regression(house_size_norm).numpy() * house_price_std) +
house_price_mean
plt.plot(house_size, pred_line, color='red', label='Fitted Line')
plt.xlabel('House Size (sq ft)')
plt.ylabel('House Price ($1000s)')
plt.title('Learned Linear Relationship')
plt.legend()
plt.show()
print(f"Trained Weight: {W.numpy()[0][0]:.4f}, Bias: {b.numpy()[0]:.4f}")
# Predictions
new_house_sizes = np.array([[750], [1500], [3000], [4500]], dtype=np.float32)
new_house_sizes_norm = (new_house_sizes - house_size_mean) / house_size_std
predicted_prices_norm = linear_regression(new_house_sizes_norm).numpy()
predicted_prices = (predicted_prices_norm * house_price_std) + house_price_mean
for size, price in zip(new_house_sizes.flatten(), predicted_prices.flatten()):
    print(f"Predicted price for house size {size} sq ft: ${price * 1000:.2f}")

# =====
# PART 2: EMNIST RNN Model
# =====

# Load dataset
(ds_train, ds_test), ds_info = tfds.load(

```

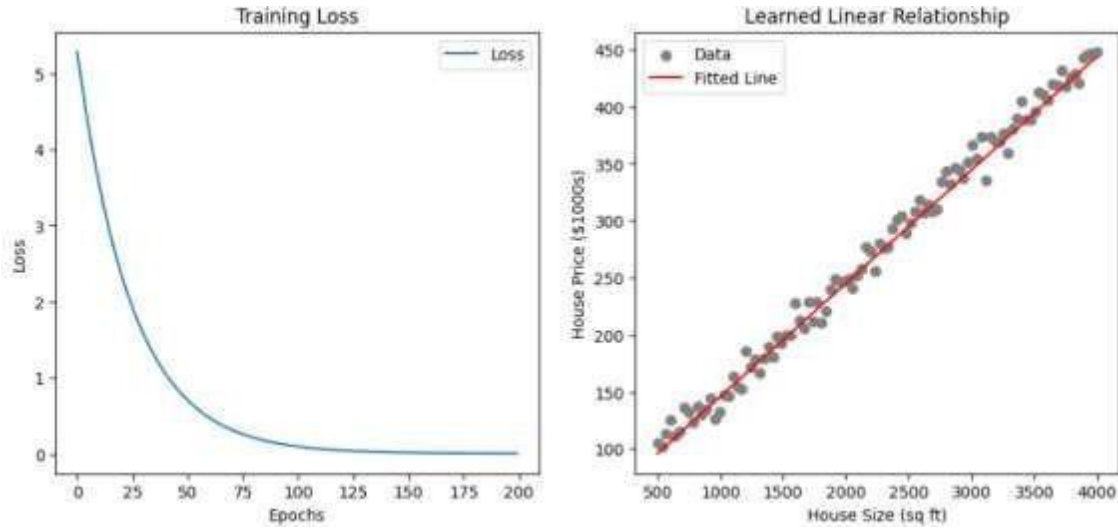
```

    'emnist/letters',
    split=['train', 'test'],
    shuffle_files=True,
    as_supervised=True,
    with_info=True
)
# Preprocessing
def preprocess(image, label):
    image = tf.cast(image, tf.float32) / 255.0
    image = tf.transpose(image)
    image = tf.image.flip_left_right(image)
    image = tf.reshape(image, (28, 28))
    label = label - 1
    return image, label
# Apply preprocessing
ds_train = ds_train.map(preprocess).batch(128).prefetch(tf.data.AUTOTUNE)
ds_test = ds_test.map(preprocess).batch(128).prefetch(tf.data.AUTOTUNE)

# Build model
model = tf.keras.Sequential([
    tf.keras.layers.SimpleRNN(128, input_shape=(28, 28)),
    tf.keras.layers.Dense(26, activation='softmax')
])
# Compile
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
# Train
model.fit(ds_train, epochs=5, validation_data=ds_test)
# Test predictions
test_images, test_labels = next(iter(ds_test))
predictions = model.predict(test_images)
# Show predictions
for i in range(5):
    plt.imshow(test_images[i], cmap='gray')
    true_label = chr(test_labels[i].numpy() + 65)
    pred_label = chr(np.argmax(predictions[i]) + 65)
    plt.title(f"True: {true_label} | Pred: {pred_label}")
    plt.axis('off')
    plt.show()

```

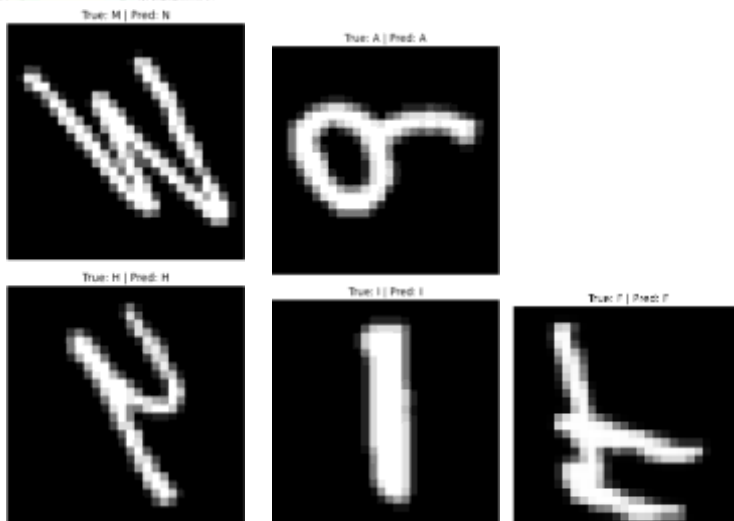
OUTPUT



```

Trained Weight: 0.0057, Bias: -0.0090
Predicted price for house size 750.0 sq ft: $120950.00
Predicted price for house size 1500.0 sq ft: $195444.52
Predicted price for house size 3000.0 sq ft: $344457.43
Predicted price for house size 4500.0 sq ft: $483470.25
WARNING: Variant folder /root/tensorflow_datasets/ewnist/letters/3.1.0 has no dataset_info.json
Downloading and preparing dataset unknown size (download: unknown size, generated: unknown size, total: unknown size) to /root/tensorflow_datasets/ewnist/letters/3.1.0...
Dl Completed... 100% 1/1 [00:20<00:00, 13.26k/s]
Dl Size... 100% 535535 [00:20<00:00, 51.22 MB/s]
Extraction completed... 100% 30/30 [00:20<00:00, 20.37s/ file]
Extraction completed... 100% 44 [00:01<00:00, 3.18 file/s]
Dataset ewnist downloaded and prepared to /root/tensorflow_datasets/ewnist/letters/3.1.0. Subsequent calls will reuse this data.
Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:189: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'input(shape)'
super().__init__(**kwargs)
694/694 ----- 35s 49ms/step - accuracy: 0.6289 - loss: 1.2922 - val_accuracy: 0.7184 - val_loss: 0.9483
Epoch 2/5
694/694 ----- 25s 35ms/step - accuracy: 0.7720 - loss: 0.7572 - val_accuracy: 0.7680 - val_loss: 0.7456
Epoch 3/5
694/694 ----- 34s 35ms/step - accuracy: 0.9090 - loss: 0.6333 - val_accuracy: 0.7861 - val_loss: 0.6781
Epoch 4/5
694/694 ----- 25s 35ms/step - accuracy: 0.8271 - loss: 0.5700 - val_accuracy: 0.8222 - val_loss: 0.6111
Epoch 5/5
694/694 ----- 24s 35ms/step - accuracy: 0.8388 - loss: 0.5307 - val_accuracy: 0.8190 - val_loss: 0.5892
4/4 ----- 0s 17ms/step

```



Practical 3

Convolutional Neural Networks (Classification)

a. Implementing deep neural network for performing binary classification task

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
# Generate Sample Data (Binary Classification)
np.random.seed(42)
x_train = np.random.rand(1000, 2)
y_train = (x_train[:, 0] + x_train[:, 1] > 1).astype(int)
x_test = np.random.rand(200, 2)
y_test = (x_test[:, 0] + x_test[:, 1] > 1).astype(int)
# Define Model
model = keras.Sequential([
    layers.Dense(16, activation='relu', input_shape=(2,)),
    layers.Dense(8, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
# Compile
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
# Train
model.fit(
    x_train, y_train,
    epochs=20,
    batch_size=32,
    validation_split=0.2
)
# Evaluate
loss, accuracy = model.evaluate(x_test, y_test)
print(f"Test Accuracy: {accuracy * 100:.2f}%")
# Predict
predictions = (model.predict(x_test) > 0.5).astype(int)
print("\nSample Predictions:")
print(predictions[:10].flatten())
```

OUTPUT

```
c:\Users\student\AppData\Local\Temp\Python\Python312\site-packages\keras\src\layers\core\dense.py:107: Overwarning: Do not pass an 'input_shape' /
input dim argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
25/25 ----- 1s 3ms/step - accuracy: 0.6625 - loss: 0.6588 - val_accuracy: 0.6590 - val_loss: 0.6576
Epoch 2/20 ----- 0s 1ms/step - accuracy: 0.6488 - loss: 0.6537 - val_accuracy: 0.7658 - val_loss: 0.6438
25/25 ----- 0s 1ms/step - accuracy: 0.7237 - loss: 0.6309 - val_accuracy: 0.7705 - val_loss: 0.6273
Epoch 3/20 ----- 0s 1ms/step - accuracy: 0.7412 - loss: 0.6133 - val_accuracy: 0.7888 - val_loss: 0.6089
25/25 ----- 0s 1ms/step - accuracy: 0.7775 - loss: 0.6047 - val_accuracy: 0.8258 - val_loss: 0.5888
Epoch 4/20 ----- 0s 1ms/step - accuracy: 0.7788 - loss: 0.5848 - val_accuracy: 0.8408 - val_loss: 0.5651
25/25 ----- 0s 2ms/step - accuracy: 0.8588 - loss: 0.5599 - val_accuracy: 0.8795 - val_loss: 0.5424
Epoch 5/20 ----- 0s 3ms/step - accuracy: 0.8737 - loss: 0.5331 - val_accuracy: 0.8755 - val_loss: 0.5126
25/25 ----- 0s 2ms/step - accuracy: 0.8737 - loss: 0.5042 - val_accuracy: 0.9088 - val_loss: 0.4832
Epoch 6/20 ----- 0s 3ms/step - accuracy: 0.8925 - loss: 0.4745 - val_accuracy: 0.9258 - val_loss: 0.4531
25/25 ----- 0s 3ms/step - accuracy: 0.9082 - loss: 0.4441 - val_accuracy: 0.9488 - val_loss: 0.4224
Epoch 7/20 ----- 0s 2ms/step - accuracy: 0.9258 - loss: 0.4141 - val_accuracy: 0.9458 - val_loss: 0.3926
25/25 ----- 0s 1ms/step - accuracy: 0.9375 - loss: 0.3851 - val_accuracy: 0.9488 - val_loss: 0.3635
Epoch 8/20 ----- 0s 2ms/step - accuracy: 0.9488 - loss: 0.3571 - val_accuracy: 0.9588 - val_loss: 0.3354
25/25 ----- 0s 1ms/step - accuracy: 0.9433 - loss: 0.3389 - val_accuracy: 0.9588 - val_loss: 0.3061
Epoch 9/20 ----- 0s 1ms/step - accuracy: 0.9425 - loss: 0.3088 - val_accuracy: 0.9588 - val_loss: 0.2854
25/25 ----- 0s 3ms/step - accuracy: 0.9463 - loss: 0.2838 - val_accuracy: 0.9588 - val_loss: 0.2638
Epoch 10/20 ----- 0s 2ms/step - accuracy: 0.9463 - loss: 0.2648 - val_accuracy: 0.9588 - val_loss: 0.2451
25/25 ----- 0s 1ms/step - accuracy: 0.9525 - loss: 0.2457 - val_accuracy: 0.9688 - val_loss: 0.2285
Epoch 11/20 ----- 0s 2ms/step - accuracy: 0.9528 - loss: 0.2388 - val_accuracy: 0.9688 - val_loss: 0.2151
25/25 ----- 0s 4ms/step - accuracy: 0.9688 - loss: 0.2121
7/7 ----- 0s 1ms/step
Test Accuracy: 98.88%
Sample Predictions:
[0 1 1 0 0 0 0 1 0]
```

b. Using a deep feed-forward network with two hidden layers for performing multiclass classification and predicting the class.

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
# Load Iris dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target.reshape(-1, 1)
# One-hot encode labels (compatible with all sklearn versions)
encoder = OneHotEncoder(sparse_output=False)
y_encoded = encoder.fit_transform(y)
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42
)
# Build model
model = keras.Sequential([
    layers.Input(shape=(4,)),
    layers.Dense(64, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(3, activation='softmax')
])
# Compile
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
# Train
model.fit(X_train, y_train, epochs=50, batch_size=16, verbose=1)
# Evaluate
loss, accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {accuracy * 100:.2f}%")
# Predict
predictions = model.predict(X_test)
predicted_classes = np.argmax(predictions, axis=1)
actual_classes = np.argmax(y_test, axis=1)
print("\nPredicted Classes:", predicted_classes)
```

```
print("Actual Classes:", actual_classes)
```

OUTPUT

```
Epoch 1/50 0/0 1s 3ms/step - accuracy: 0.3750 - loss: 1.0976
Epoch 2/50 0/0 2ms/step - accuracy: 0.7000 - loss: 0.9530
Epoch 3/50 0/0 0s 2ms/step - accuracy: 0.7500 - loss: 0.8861
Epoch 4/50 0/0 4ms/step - accuracy: 0.8417 - loss: 0.8206
Epoch 5/50 0/0 0s 2ms/step - accuracy: 0.7833 - loss: 0.7698
Epoch 6/50 0/0 0s 3ms/step - accuracy: 0.8667 - loss: 0.7222
Epoch 7/50 0/0 0s 2ms/step - accuracy: 0.7333 - loss: 0.6773
Epoch 8/50 0/0 0s 2ms/step - accuracy: 0.7167 - loss: 0.6325
Epoch 9/50 0/0 0s 2ms/step - accuracy: 0.8000 - loss: 0.5941
Epoch 10/50 0/0 0s 2ms/step - accuracy: 0.7000 - loss: 0.5761
Epoch 11/50 0/0 0s 3ms/step - accuracy: 0.8000 - loss: 0.5382
Epoch 12/50 0/0 0s 2ms/step - accuracy: 0.8500 - loss: 0.5082
Epoch 13/50 0/0 0s 2ms/step - accuracy: 0.8000 - loss: 0.4540
Epoch 14/50 0/0 0s 2ms/step - accuracy: 0.9000 - loss: 0.4078
Epoch 15/50 0/0 0s 2ms/step - accuracy: 0.9250 - loss: 0.4000
Epoch 16/50 0/0 0s 2ms/step - accuracy: 0.8000 - loss: 0.4330
Epoch 17/50 0/0 0s 3ms/step - accuracy: 0.9417 - loss: 0.4359
Epoch 18/50 0/0 0s 2ms/step - accuracy: 0.9500 - loss: 0.4011
Epoch 19/50 0/0 0s 4ms/step - accuracy: 0.9500 - loss: 0.3878
Epoch 20/50 0/0 0s 4ms/step - accuracy: 0.9500 - loss: 0.3747
Epoch 21/50 0/0 0s 4ms/step - accuracy: 0.9667 - loss: 0.3611
Epoch 22/50 0/0 0s 4ms/step - accuracy: 0.9500 - loss: 0.3514
Epoch 23/50 0/0 0s 2ms/step - accuracy: 0.9667 - loss: 0.3376
Epoch 24/50 0/0 0s 3ms/step - accuracy: 0.9250 - loss: 0.3200
Epoch 25/50 0/0 0s 2ms/step - accuracy: 0.9667 - loss: 0.3104
Epoch 26/50 0/0 0s 2ms/step - accuracy: 0.9583 - loss: 0.3020
Epoch 27/50 0/0 0s 1ms/step - accuracy: 0.9250 - loss: 0.3014
Epoch 28/50 0/0 0s 0ms/step - accuracy: 0.9667 - loss: 0.2984
Epoch 29/50 0/0 0s 2ms/step - accuracy: 0.9500 - loss: 0.2760
Epoch 30/50 0/0 0s 3ms/step - accuracy: 0.9567 - loss: 0.2018
Epoch 31/50 0/0 0s 2ms/step - accuracy: 0.9667 - loss: 0.2523
Epoch 32/50 0/0 0s 2ms/step - accuracy: 0.9667 - loss: 0.2423
Epoch 33/50 0/0 0s 2ms/step - accuracy: 0.9667 - loss: 0.2380
Epoch 34/50 0/0 0s 2ms/step - accuracy: 0.9583 - loss: 0.2251
Epoch 35/50 0/0 0s 2ms/step - accuracy: 0.9567 - loss: 0.2106
Epoch 36/50 0/0 0s 1ms/step - accuracy: 0.9583 - loss: 0.2000
Epoch 37/50 0/0 0s 2ms/step - accuracy: 0.9567 - loss: 0.1996
Epoch 38/50 0/0 0s 2ms/step - accuracy: 0.9750 - loss: 0.1912
Epoch 39/50 0/0 0s 2ms/step - accuracy: 0.9583 - loss: 0.1801
Epoch 40/50 0/0 0s 2ms/step - accuracy: 0.9583 - loss: 0.1801
Epoch 41/50 0/0 0s 2ms/step - accuracy: 0.9500 - loss: 0.1792
Epoch 42/50 0/0 0s 2ms/step - accuracy: 0.9583 - loss: 0.1773
Epoch 43/50 0/0 0s 2ms/step - accuracy: 0.9667 - loss: 0.1624
Epoch 44/50 0/0 0s 2ms/step - accuracy: 0.9583 - loss: 0.1606
Epoch 45/50 0/0 0s 2ms/step - accuracy: 0.9583 - loss: 0.1525
Epoch 46/50 0/0 0s 2ms/step - accuracy: 0.9667 - loss: 0.1525
Epoch 47/50 0/0 0s 2ms/step - accuracy: 0.9500 - loss: 0.1545
Epoch 48/50 0/0 0s 2ms/step - accuracy: 0.9583 - loss: 0.1513
Epoch 49/50 0/0 0s 4ms/step - accuracy: 0.9583 - loss: 0.1403
Epoch 50/50 0/0 0s 2ms/step - accuracy: 0.9667 - loss: 0.1235
1/1 0s 95ms/step - accuracy: 1.0000 - loss: 0.1239
Test Accuracy: 100.00%
1/1 0s 32ms/step
Predicted Classes: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Actual Classes: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
```

4 Write a program to implement deep learning techniques for image segmentation.

```
# !pip install tensorflow matplotlib
# Imports
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
# =====
# Step 1: Generate Data
# =====
def generate_synthetic_data(num_samples=100, image_size=64):
    images = np.zeros((num_samples, image_size, image_size, 1), dtype=np.float32)
    masks = np.zeros((num_samples, image_size, image_size, 1), dtype=np.float32)
    for i in range(num_samples):
        radius = np.random.randint(5, 15)
        center_x = np.random.randint(radius, image_size - radius)
        center_y = np.random.randint(radius, image_size - radius)
        y, x = np.ogrid[:image_size, :image_size]
        mask = (x - center_x) ** 2 + (y - center_y) ** 2 <= radius ** 2
        noise = 0.1 * np.random.randn(image_size, image_size)
        images[i, ..., 0] = mask.astype(np.float32) + noise
        masks[i, ..., 0] = mask.astype(np.float32)
    return images, masks
# Generate dataset
X, y = generate_synthetic_data(200)
# Split data
X_train, X_test = X[:160], X[160:]
y_train, y_test = y[:160], y[160:]
# =====
# Step 2: Build U-Net Model
# =====
def build_unet(input_shape=(64, 64, 1)):
    inputs = layers.Input(shape=input_shape)
    # Encoder
    c1 = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(inputs)
    c1 = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(c1)
    p1 = layers.MaxPooling2D((2, 2))(c1)

    c2 = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(p1)
    c2 = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(c2)
    p2 = layers.MaxPooling2D((2, 2))(c2)
    # Bottleneck
    c3 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(p2)
```



```

c3 = layers.Conv2D(64, (3, 3), activation='relu', padding='same')(c3)
# Decoder
u1 = layers.UpSampling2D((2, 2))(c3)
u1 = layers.Concatenate()([u1, c2])
c4 = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(u1)
c4 = layers.Conv2D(32, (3, 3), activation='relu', padding='same')(c4)
u2 = layers.UpSampling2D((2, 2))(c4)
u2 = layers.Concatenate()([u2, c1])
c5 = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(u2)
c5 = layers.Conv2D(16, (3, 3), activation='relu', padding='same')(c5)
outputs = layers.Conv2D(1, (1, 1), activation='sigmoid')(c5)
model = models.Model(inputs, outputs)
return model
# =====
# Step 3: Compile & Train
# =====
model = build_unet()
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=16,
    validation_split=0.1
)
# =====
# Step 4: Visualization
# =====
def visualize_predictions(images, masks, predictions, num=5):
    plt.figure(figsize=(12, num * 3))
    for i in range(num):
        # Input
        plt.subplot(num, 3, i * 3 + 1)
        plt.title("Input")
        plt.imshow(images[i].squeeze(), cmap='gray')
        plt.axis('off')

        # Ground Truth
        plt.subplot(num, 3, i * 3 + 2)
        plt.title("Ground Truth")
        plt.imshow(masks[i].squeeze(), cmap='gray')

```

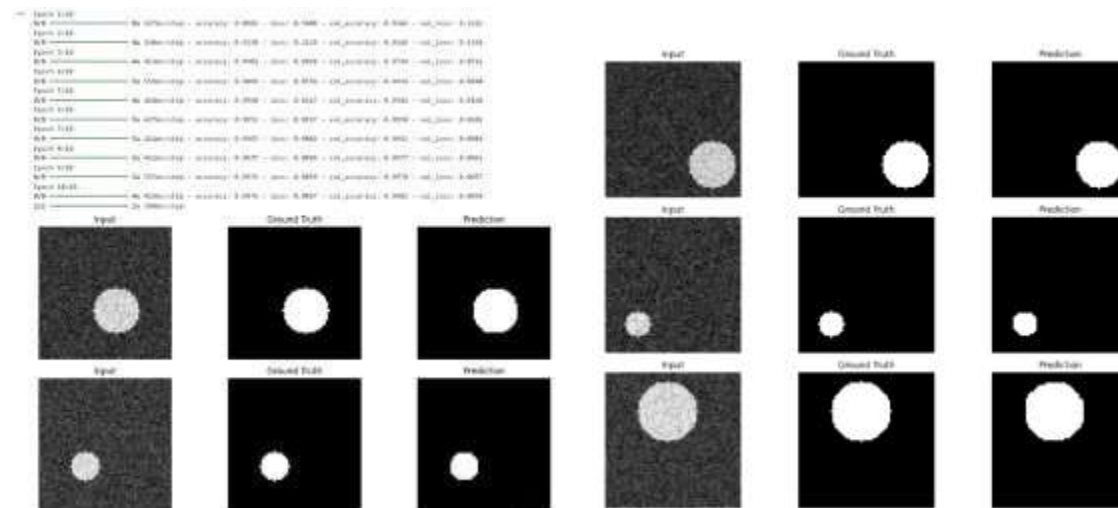
```

plt.axis('off')

# Prediction
plt.subplot(num, 3, i * 3 + 3)
plt.title("Prediction")
plt.imshow((predictions[i].squeeze() > 0.5), cmap='gray')
plt.axis('off')
plt.tight_layout()
plt.show()
# =====
# Step 5: Predict & Show
# =====
preds = model.predict(X_test)
visualize_predictions(X_test, y_test, preds)

```

OUTPUT



Practical 5

Write a program to predict a caption for a sample image using LSTM.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import (
    Input,
    Dense,
    LSTM,
    Embedding,
    Add
)
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.preprocessing.image import load_img, img_to_array
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input
# -----
# Small Dataset
# -----
images = [
    "/content/dog.jpg",
    "/content/cat.jpg",
    "/content/bike.webp"
]
captions = [
    "startseq a dog running in the park endseq",
    "startseq a cat sitting on a couch endseq",
    "startseq a person riding a bike endseq"
]

# -----
# VGG16 Feature Extractor
# -----
base_model = VGG16(weights="imagenet")
model_vgg = Model(
    inputs=base_model.input,
    outputs=base_model.layers[-2].output
)
def extract_features(image_path):
    img = load_img(image_path, target_size=(224, 224))
    img_array = img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
```

```

img_array = preprocess_input(img_array)
features = model_vgg.predict(img_array, verbose=0)
return features
# -----
# Tokenize Captions
# -----
tokenizer = Tokenizer()
tokenizer.fit_on_texts(captions)
max_length = max(len(caption.split()) for caption in captions)
vocab_size = len(tokenizer.word_index) + 1
sequences = tokenizer.texts_to_sequences(captions)
padded_captions = pad_sequences(
    sequences,
    maxlen=max_length,
    padding="post"
)
# -----
# Caption Generation Model
# -----
embedding_dim = 256
hidden_units = 256
# Image branch
image_input = Input(shape=(4096,))
image_output = Dense(
    embedding_dim,
    activation="relu"
)(image_input)
# Text branch
caption_input = Input(shape=(max_length,))
embedding = Embedding(
    input_dim=vocab_size,
    output_dim=embedding_dim,
    mask_zero=True
)(caption_input)
lstm_out = LSTM(hidden_units)(embedding)
# Combine image and text features
combined = Add()([image_output, lstm_out])
decoder_output = Dense(
    vocab_size,
    activation="softmax"
)(combined)
model = Model(
    inputs=[image_input, caption_input],
    outputs=decoder_output

```

```

)
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy"
)
model.summary()
# -----
# Dummy Training Labels
# -----
y_train = np.zeros((3, vocab_size), dtype=np.float32)
model.fit(
    [np.random.rand(3, 4096), padded_captions],
    y_train,
    epochs=5,
    batch_size=1,
    verbose=1
)
# -----
# Caption Generation Function
# -----
def generate_caption(model, tokenizer, image_feature, max_length):
    in_text = "startseq"
    for _ in range(max_length):
        sequence = tokenizer.texts_to_sequences([in_text])[0]
        sequence = pad_sequences(
            [sequence],
            maxlen=max_length,
            padding="post"
        )
        prediction = model.predict(
            [image_feature, sequence],
            verbose=0
        )
        yhat = np.argmax(prediction)
        word = tokenizer.index_word.get(yhat)
        if word is None:
            break
        in_text += " " + word
        if word == "endseq":
            break
    return (
        in_text
        .replace("startseq", "")
        .replace("endseq", "")

```

```

        .strip()
    )
# -----
# Test on Sample Image
# -----
sample_image_path = images[0]
plt.imshow(load_img(sample_image_path))
plt.axis("off")
plt.show()
sample_feature = extract_features(sample_image_path)
caption = generate_caption(
    model,
    tokenizer,
    sample_feature,
    max_length
)
print("Predicted Caption:", caption)

```

OUTPUT

Model: "functional_3"

Layer (type)	Output Shape	Param #	Connected to
input_layer_5 (InputLayer)	(None, 8)	0	-
input_layer_4 (InputLayer)	(None, 4096)	0	-
embedding_1 (Embedding)	(None, 8, 256)	4,096	input_layer_5[0]...
not_equal_1 (NotEqual)	(None, 8)	0	input_layer_5[0]...
dense_2 (Dense)	(None, 256)	1,048,632	input_layer_4[0]...
lstm_1 (LSTM)	(None, 256)	525,312	embedding_1[0][0]..., not_equal_1[0][0]
add_1 (Add)	(None, 256)	0	dense_2[0][0], lstm_1[0][0]
dense_3 (Dense)	(None, 16)	4,112	add_1[0][0]

Total params: 1,582,352 (6.04 MB)
 Trainable params: 1,582,352 (6.04 MB)
 Non-trainable params: 0 (0.00 B)

Epoch 1/5
 3/3 ————— 3s 35ms/step - loss: 0.0000e+00
 Epoch 2/5
 3/3 ————— 0s 37ms/step - loss: 0.0000e+00
 Epoch 3/5
 3/3 ————— 0s 33ms/step - loss: 0.0000e+00
 Epoch 4/5
 3/3 ————— 0s 33ms/step - loss: 0.0000e+00
 Epoch 5/5
 3/3 ————— 0s 33ms/step - loss: 0.0000e+00



Predicted Caption: in in in in in in in in

Practical 6

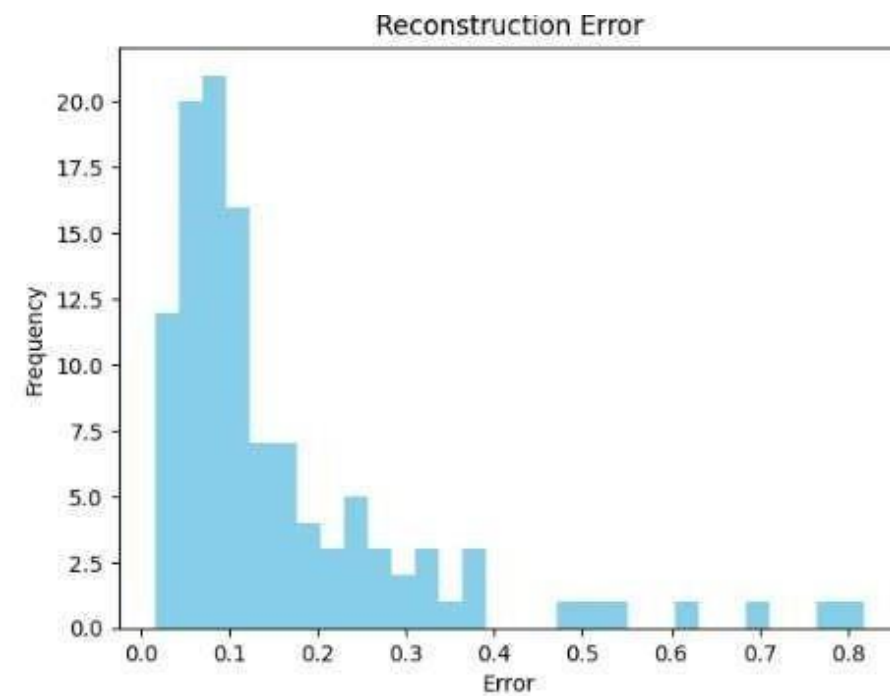
Applying the Autoencoder algorithms for encoding real-world data.

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
# Load dataset
data = load_breast_cancer()
X = data.data
# Normalize
X = StandardScaler().fit_transform(X)
# Train-test split
X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)
# Dimensions
input_dim = X.shape[1]
encoding_dim = 10
# Input layer
input_layer = Input(shape=(input_dim,))
# Encoder
encoded = Dense(encoding_dim, activation='relu')(input_layer)
# Decoder
decoded = Dense(input_dim, activation='linear')(encoded)
# Autoencoder model
autoencoder = Model(inputs=input_layer, outputs=decoded)
# Compile
autoencoder.compile(optimizer='adam', loss='mse')
# Train
history = autoencoder.fit(
    X_train, X_train,
    epochs=50,
    batch_size=16,
    validation_data=(X_test, X_test),
    verbose=0
)
# Separate encoder model (IMPORTANT)
encoder = Model(inputs=input_layer, outputs=encoded)
# Encode + Decode
encoded_data = encoder.predict(X_test)
```

```
reconstructed_data = autoencoder.predict(X_test)
# Reconstruction error
reconstruction_error = np.mean(np.square(X_test - reconstructed_data), axis=1)
# Plot error
plt.hist(reconstruction_error, bins=30, color='skyblue')
plt.title("Reconstruction Error")
plt.xlabel("Error")
plt.ylabel("Frequency")
plt.show()
print("Encoded shape:", encoded_data.shape)
```

OUTPUT



Encoded shape: (114, 10)

Practical 7

Write a program for character recognition using RNN and compare it with CNN.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Dense,
    LSTM,
    Conv2D,
    MaxPooling2D,
    Flatten,
    Dropout
)
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
# Load and preprocess the MNIST dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
# Reshape data for CNN
x_train_cnn = x_train.reshape(-1, 28, 28, 1)
x_test_cnn = x_test.reshape(-1, 28, 28, 1)

# Reshape data for RNN
x_train_rnn = x_train.reshape(-1, 28, 28)
x_test_rnn = x_test.reshape(-1, 28, 28)
# One-hot encode the labels
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
# Define RNN model
rnn_model = Sequential([
    LSTM(128, input_shape=(28, 28)),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])
rnn_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
# Define CNN model
cnn_model = Sequential([
```

```

Conv2D(
    32,
    kernel_size=(3, 3),
    activation='relu',
    input_shape=(28, 28, 1)
),
MaxPooling2D(pool_size=(2, 2)),
Conv2D(
    64,
    kernel_size=(3, 3),
    activation='relu'
),
MaxPooling2D(pool_size=(2, 2)),
Flatten(),
Dense(128, activation='relu'),
Dropout(0.5),
Dense(10, activation='softmax')
])

cnn_model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
# Train the RNN model
print("Training RNN model...")
rnn_history = rnn_model.fit(
    x_train_rnn,
    y_train,
    epochs=10,
    batch_size=128,
    validation_data=(x_test_rnn, y_test)
)
# Train the CNN model
print("Training CNN model...")
cnn_history = cnn_model.fit(
    x_train_cnn,
    y_train,
    epochs=10,
    batch_size=128,
    validation_data=(x_test_cnn, y_test)
)
# Evaluate the models
rnn_score = rnn_model.evaluate(

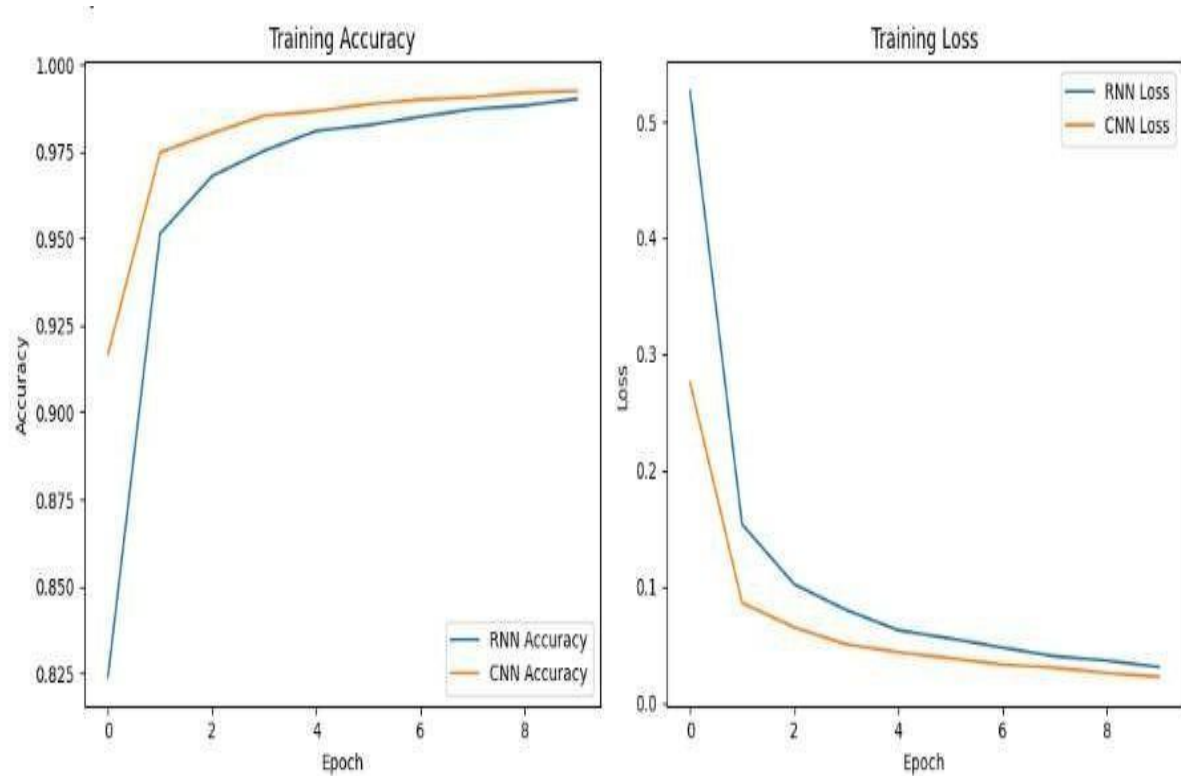
```

```

    x_test_rnn,
    y_test,
    verbose=0
)
cnn_score = cnn_model.evaluate(
    x_test_cnn,
    y_test,
    verbose=0
)
print(f'RNN Accuracy: {rnn_score[1] * 100:.2f}%')
print(f'CNN Accuracy: {cnn_score[1] * 100:.2f}%')
# Plot training accuracy and loss
plt.figure(figsize=(12, 5))
# Accuracy
plt.subplot(1, 2, 1)
plt.plot(
    rnn_history.history['accuracy'],
    label='RNN Accuracy'
)
plt.plot(
    cnn_history.history['accuracy'],
    label='CNN Accuracy'
)
plt.title('Training Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
# Loss
plt.subplot(1, 2, 2)
plt.plot(
    rnn_history.history['loss'],
    label='RNN Loss'
)
plt.plot(
    cnn_history.history['loss'],
    label='CNN Loss'
)
plt.title('Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.tight_layout()
plt.show()

```

OUTPUT



Practical 8

Write a program to develop Autoencoders using MNIST Handwritten Digits .

```
# Step 1: Import necessary libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten, Reshape
from tensorflow.keras.datasets import mnist

# Load and preprocess the MNIST dataset
(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))

# Define the Autoencoder architecture
input_img = Input(shape=(28, 28, 1))

# Encoder
x = Flatten()(input_img)
x = Dense(128, activation='relu')(x)
x = Dense(64, activation='relu')(x)
encoded = Dense(32, activation='relu')(x)

# Decoder
x = Dense(64, activation='relu')(encoded)
x = Dense(128, activation='relu')(x)
x = Dense(28 * 28, activation='sigmoid')(x)
decoded = Reshape((28, 28, 1))(x)

# Define the Autoencoder model
autoencoder = Model(inputs=input_img, outputs=decoded)

# Compile the model
autoencoder.compile(
    optimizer='adam',
    loss='binary_crossentropy'
)

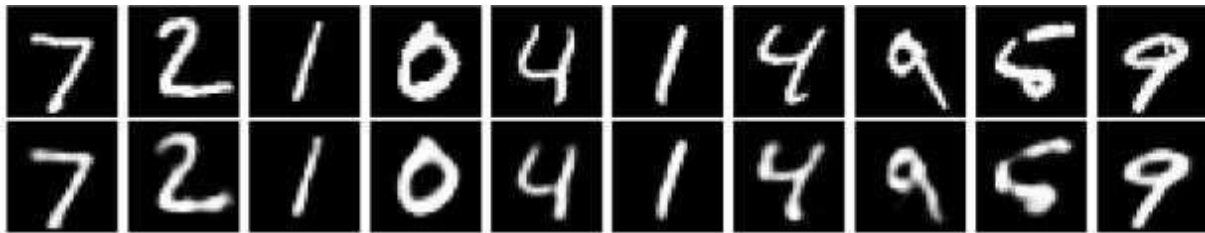
# Train the Autoencoder
autoencoder.fit(
    x_train,
    x_train,
    epochs=50,
    batch_size=256,
    shuffle=True,
    validation_data=(x_test, x_test)
)
```

```

# Reconstruct test images
reconstructed_imgs = autoencoder.predict(x_test)
# Display original and reconstructed images
n = 10
plt.figure(figsize=(20, 4))
for i in range(n):
    # Original image
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
    # Reconstructed image
    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(reconstructed_imgs[i].reshape(28, 28), cmap='gray')
    plt.axis('off')
plt.tight_layout()
plt.show()

```

OUTPUT



Practical 9

Demonstrate recurrent neural network that learns to perform sequence analysis for stock price.(google stock price)

```
# Step 1: Install and import libraries
# !pip install yfinance --quiet # uncomment in Jupyter/Colab
import yfinance as yf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# Step 2: Download Google stock data
df = yf.download('GOOG', start='2020-01-01', end='2024-12-31')
df = df[['Open', 'Close', 'High', 'Low']]

# Step 3: Normalize data
scaler = MinMaxScaler()
scaled_data = scaler.fit_transform(df)

# Step 4: Create sequences
def create_sequences(data, seq_length=60):
    x, y = [], []
    for i in range(len(data) - seq_length):
        x.append(data[i:i + seq_length])
        y.append(data[i + seq_length])
    return np.array(x), np.array(y)

sequence_length = 60
X, y = create_sequences(scaled_data, sequence_length)

# Step 5: Split data
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# Step 6: Build LSTM model
model = Sequential([
    LSTM(64, return_sequences=True, input_shape=(X_train.shape[1], X_train.shape[2])),
    LSTM(32),
    Dense(4) # Open, Close, High, Low
])

model.compile(optimizer='adam', loss='mse')
model.summary()

# Step 7: Train model
history = model.fit(
```

```

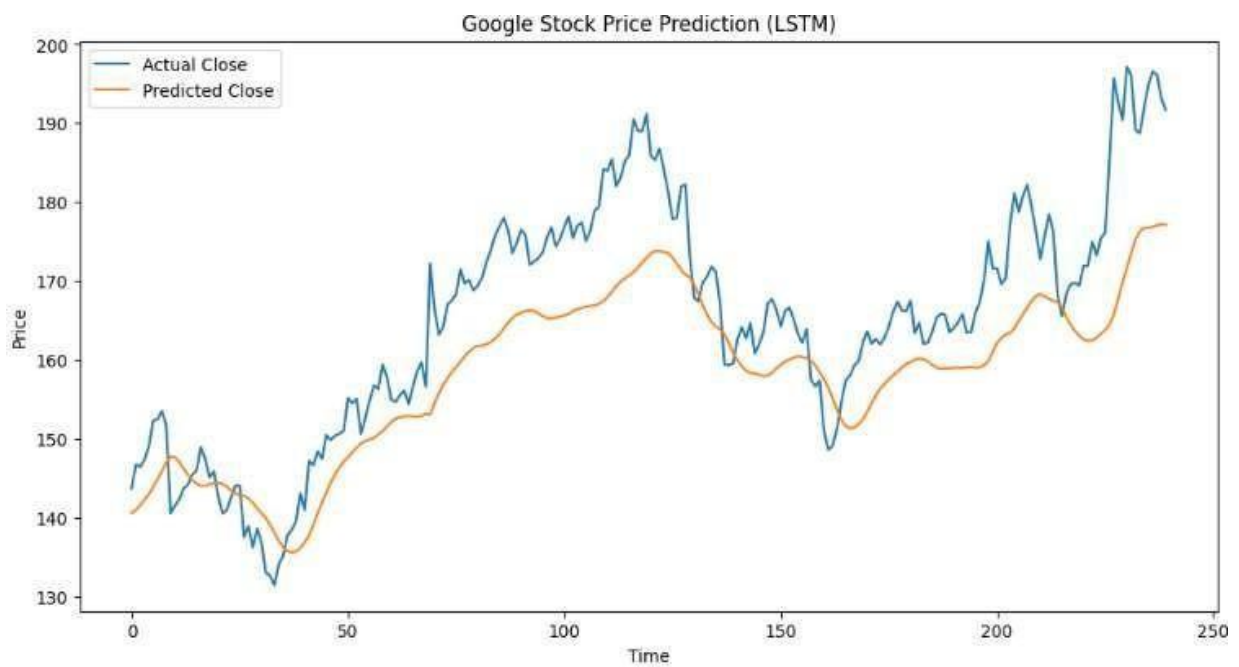
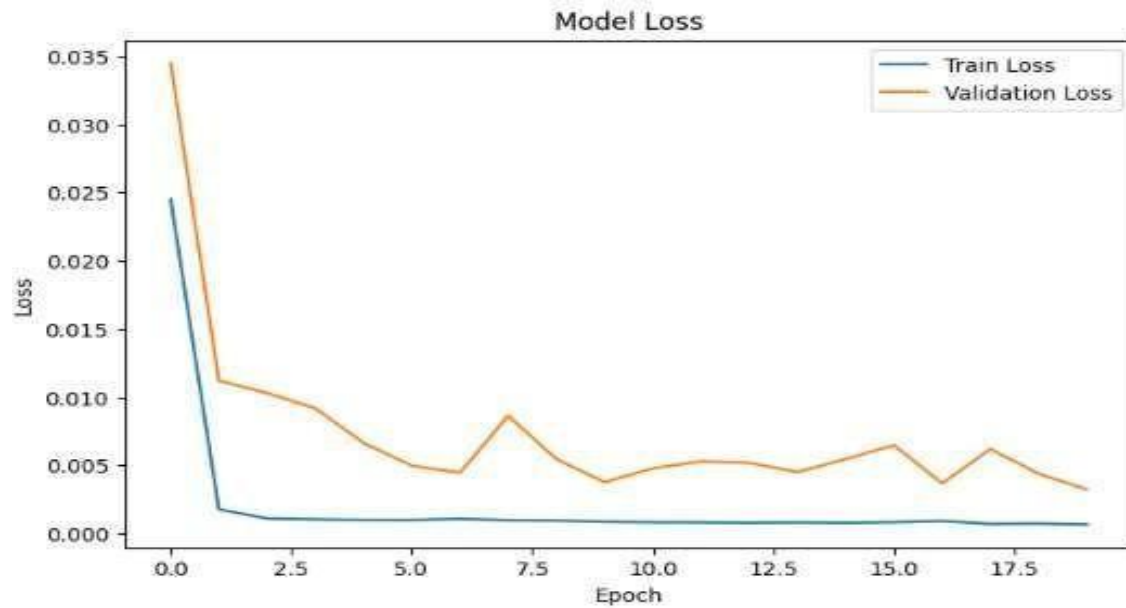
X_train, y_train,
epochs=20,
batch_size=32,
validation_data=(X_test, y_test)
)
# Step 8: Plot training history
plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
# Step 9: Predictions
predicted = model.predict(X_test)
predicted_prices = scaler.inverse_transform(predicted)
actual_prices = scaler.inverse_transform(y_test)
# Step 10: Plot results
plt.figure(figsize=(12,6))
plt.plot(actual_prices[:, 1], label='Actual Close')
plt.plot(predicted_prices[:, 1], label='Predicted Close')
plt.title("Google Stock Price Prediction (LSTM)")
plt.xlabel("Time")
plt.ylabel("Price")
plt.legend()
plt.show()

```

OUTPUT

Model: "sequential_3"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 64)	17,664
lstm_1 (LSTM)	(None, 32)	12,416
dense_14 (Dense)	(None, 4)	132



Practical 10

Applying Generative Adversarial Networks for image generation and unsupervised.

```
import tensorflow as tf
from tensorflow.keras.layers import (
    Dense, Reshape, Flatten, LeakyReLU, Dropout,
    BatchNormalization, Conv2DTranspose, Conv2D
)
from tensorflow.keras.models import Sequential
import numpy as np
import matplotlib.pyplot as plt
# -----
# Generator
# -----
def build_generator():
    model = Sequential()

    model.add(Dense(7 * 7 * 256, input_dim=100))
    model.add(LeakyReLU(0.2))
    model.add(Reshape((7, 7, 256)))

    model.add(Conv2DTranspose(128, kernel_size=4, strides=2, padding='same'))
    model.add(BatchNormalization())
    model.add(LeakyReLU(0.2))

    model.add(Conv2DTranspose(64, kernel_size=4, strides=2, padding='same'))
    model.add(BatchNormalization())
    model.add(LeakyReLU(0.2))
    model.add(Conv2D(1, kernel_size=7, padding='same', activation='tanh'))
    return model
# -----
# Discriminator
# -----
def build_discriminator():
    model = Sequential()
    model.add(Conv2D(64, kernel_size=3, strides=2, padding='same',
        input_shape=(28, 28, 1)))
    model.add(LeakyReLU(0.2))
    model.add(Dropout(0.3))
    model.add(Conv2D(128, kernel_size=3, strides=2, padding='same'))
    model.add(LeakyReLU(0.2))
    model.add(Dropout(0.3))
    model.add(Flatten())
    model.add(Dense(1, activation='sigmoid'))
```

```

    return model
# -----
# GAN Model
# -----
def build_gan(generator, discriminator):
    discriminator.compile(
        loss='binary_crossentropy',
        optimizer='adam',
        metrics=['accuracy']
    )
    discriminator.trainable = False

    model = Sequential([generator, discriminator])
    model.compile(
        loss='binary_crossentropy',
        optimizer='adam'
    )
    return model

# -----
# Load MNIST Dataset
# -----
(X_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
X_train = X_train.astype(np.float32)
X_train = (X_train / 127.5) - 1.0
X_train = np.expand_dims(X_train, axis=-1)
# -----
# Sample Images Function
# -----
def sample_images(generator, rows=4, cols=4):
    noise = np.random.normal(0, 1, (rows * cols, 100))
    gen_imgs = generator.predict(noise, verbose=0)
    gen_imgs = 0.5 * gen_imgs + 0.5 # rescale to [0,1]
    fig, axs = plt.subplots(rows, cols, figsize=(4, 4))
    count = 0
    for i in range(rows):
        for j in range(cols):
            axs[i, j].imshow(gen_imgs[count, :, :, 0], cmap='gray')
            axs[i, j].axis('off')
            count += 1

    plt.show()
# -----
# Training Function

```

```

# -----
def train_gan(gan, generator, discriminator,
              epochs=100, batch_size=64, sample_interval=20):
    valid = np.ones((batch_size, 1))
    fake = np.zeros((batch_size, 1))
    for epoch in range(epochs):
        # Train Discriminator
        idx = np.random.randint(0, X_train.shape[0], batch_size)
        real_imgs = X_train[idx]

        noise = np.random.normal(0, 1, (batch_size, 100))
        fake_imgs = generator.predict(noise, verbose=0)

        d_loss_real = discriminator.train_on_batch(real_imgs, valid)
        d_loss_fake = discriminator.train_on_batch(fake_imgs, fake)

        d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

        # Train Generator
        noise = np.random.normal(0, 1, (batch_size, 100))
        g_loss = gan.train_on_batch(noise, valid)

        # Print progress
        if epoch % sample_interval == 0:
            print(f'Epoch {epoch} | D Loss: {d_loss[0]:.4f} | G Loss: {g_loss:.4f}')
            sample_images(generator)

# -----
# Build Models
# -----
generator = build_generator()
discriminator = build_discriminator()
gan = build_gan(generator, discriminator)
# -----
# Train GAN
# -----
train_gan(gan, generator, discriminator)

```

OUTPUT

Epoch 80 | D Loss: 0.7881 | G Loss: 0.3649

